

0-k BFS – Speaker Script

Companion text to presentation_en.pdf

Artemov Makar, Mark Shkut
Skoltech, Algorithms 2026

May 21, 2026

The text below is the literal script – what to say on each slide. Each section is one slide; read the paragraph aloud and click forward when you hit the dashed line.

1. Title

“Good morning. I’m Makar, this is Mark. We’ve built a C++17 implementation of the 0-k BFS family of shortest-path algorithms and benchmarked it against Dijkstra. The headline number is that on a graph with nine hundred million vertices, our algorithm runs in two minutes where Dijkstra takes ten. I’ll explain the algorithm, show it running, and walk through the measurements. About fifteen minutes plus questions.” _____

2. The problem

“The problem is single-source shortest path on a weighted graph. The twist is that the weights are non-negative integers bounded by a small known constant k . This restriction is not artificial – it comes up constantly in grid path planning, in layered graphs, in 2-SAT-style reductions, in transport networks with a few toll levels. Our concrete target was to push the input size as close to one billion vertices as the hardware allows, on a single workstation.” _____

3. Baseline: Dijkstra

“The classical answer is Dijkstra. We use the textbook implementation with a `std::priority_queue` of (distance, vertex) pairs and lazy decrease-key – whenever we improve a distance we push a fresh entry, and we skip stale entries when we pop them. Complexity $O((V + E) \log V)$. This works for any non-negative real weights. The $\log V$ factor sits inside every successful relaxation – it’s the cost of the heap. The question is, can we get rid of it when the weights are small integers?” _____

4. The trick: 0-1 BFS

“The simplest case is weights zero or one. We replace the priority queue with a deque. An edge of weight zero pushes to the front, an edge of weight one pushes to the back. The deque is always sorted by distance and contains at most two distinct values at any time – the current layer at the front, the next layer at the back. Linear time, linear memory. One subtle point: unlike plain BFS, a vertex can be discovered several times before its distance is final, so the relaxation check must be ‘dist[v] plus weight is less than dist[u]’, not just ‘visited or not’.” _____

5. Generalize: 0-k BFS

“Now generalize to weights up to k . The observation is that a relaxation can move a vertex at most k steps forward in distance order. So at any moment, only $k + 1$ distance layers are alive. We allocate $k + 1$ FIFO queues and index them by distance modulo $k + 1$ – that’s the circular buffer of buckets. A pointer `cur` walks through them. Relaxation pushes the target into the bucket for the new distance. When the active bucket empties, `cur` advances. Stale entries from earlier relaxations sit in their old buckets and get skipped on pop. Complexity $O(kV + E)$, memory $O(k + E)$ for the queues plus $O(V)$ for the distance array.” _____

6. What it looks like

“Here are both algorithms running on the same grid – two hundred by two hundred, $k = 8$. They produce the same distance field – the wavefront spreads radially from the centre, shaped by random terrain weights. The titles show the actual measured wall-clock times. The left panel, 0- k BFS, finishes in four milliseconds. The right panel, Dijkstra, takes thirteen point three milliseconds – about three and a third times slower. Same correctness, much less time. The animation itself is in the repository; this is one frame from it.”

7. Under the hood

“This is the pedagogical heart. On a tiny grid – ten by ten, $k = 3$ – we instrumented the algorithm to dump every event. On the right you see the four real queues, $Q[0]$ through $Q[3]$, with their current contents. The numbers inside the cells are vertex ids. On the left is the grid; each cell’s colour shows which bucket currently owns that vertex. The red ‘cur’ arrow marks the active bucket. Watch how a relaxation lands a vertex in the bucket chosen by the new distance modulo four, and how the active bucket drains in FIFO order before `cur` advances. That’s the whole secret of the algorithm.”

8. Designing for $V \rightarrow 10^9$

“Now the engineering. To hit one billion vertices we cannot store edges explicitly – a CSR with four edges per vertex and 32-bit ids would already cost twenty gigabytes. So we built two backends. The general `GraphCSR` for ordinary graphs, and an implicit `GridGraph` that computes the four neighbours of each cell on the fly with no edge storage. Second, we made the distance type `uint32_t` instead of `uint64_t` – that halves the dominant memory cost. The trade-off is only valid as long as k times V fits in a 32-bit integer, which is fine for everything we care about and we have a 64-bit fallback.”

9. Results

“Here are the numbers, all grid, single thread. At a million vertices we already see a $4.6\times$ speed-up. At ten million, $5.6\times$. At one hundred million with $k = 1$, the deque-based 0-1 BFS reaches an eleven times speed-up – this is the peak. The next row is interesting: at half a billion vertices both algorithms become memory-bandwidth bound and the gap collapses to about one. The bottom row is the headline – on nine hundred million vertices, 0-1 BFS does the work in one hundred twenty-one seconds where Dijkstra takes six hundred twenty. Every single distance checksum matches across algorithms – nineteen distinct graph instances in the validation sweep, zero mismatches.”

10. Scaling

“Same data, log-log plot. Both axes logarithmic. Dijkstra is the top line, the BFS variants run parallel underneath. The constant vertical gap on a log scale is a constant multiplicative speed-up. Towards the right side, at 10^8 and beyond, the lines start to converge – we are entering the memory-bound regime. The lowest line throughout is the deque-based 0-1 BFS, which stays cleanly ahead.”

11. Limits

“Honest about the boundaries. First: this is a constant-factor win, not asymptotic. Once k grows comparable to $\log V$ Dijkstra is at least as good. Second: above half a billion vertices everything is memory-bound, and the queue overhead actually starts to hurt. Third: maintaining $k + 1$ separate FIFO allocators is more expensive for the cache than a single deque – that’s why our 0-1 BFS at $V \approx 10^9$ still wins but a generic 0- k BFS at the same scale would not. Fourth: this isn’t a replacement for Dijkstra in general – for real-valued weights or an unknown weight range, use Dijkstra.”

12. Next steps

“Four planned items. First, a proper indexed d -ary heap with real decrease-key for the baseline – to be a fairer reference. Second, a chunked ring-buffer FIFO for 0- k BFS to close the cache gap against 0-1 BFS. Third, leave the synthetic-graph comfort zone and load a real road network – DIMACS USA with weights quantised to $k = 8$. Fourth, a stretch goal: a Δ -stepping variant for parallel context.”

13. Thank you

“All of this – code, the LaTeX report, the animations, reproducible build scripts – lives in the GitHub repository linked here. Happy to take questions.”

Likely questions:

- *Why is 0-1 BFS faster than 0- k BFS at $k = 1$?* – One `std::deque` versus two `std::queues`: allocator and cache overhead.
- *How did you verify correctness on a billion-vertex run?* – Distance-sum checksums match Dijkstra exactly; the sum pins down the array without per-vertex comparison.
- *What about parallelism?* – Single thread throughout Stage 1; Δ -stepping is planned.